
智洁园区 · 废气综合管理平台

系统部署手册

含 Docker 部署步骤 · 环境配置要求 · 依赖包清单 · 常见问题排查

文档版本 v1.0

发布日期 2026-04-21

适用版本 admin-frontend 1.0.0 / admin-backend 0.1.0

编写团队 智洁园区项目组

目录

1	项目概述	2
1.1	系统简介	2
1.2	子系统构成	2
1.3	数据流概览	2
2	环境配置要求	2
2.1	硬件最低配置	2
2.2	操作系统与基础软件	3
3	依赖包清单	3
3.1	ML 推理服务（根目录 requirements.txt）	3
3.2	管理端后端（admin/backend/requirements.txt）	4
3.3	管理端前端（admin/frontend/package.json）	4
4	Docker 部署步骤	5
4.1	目录准备	5
4.2	编写/确认环境变量文件	5
4.3	一键启动全部服务	6
4.4	管理端独立镜像（推荐扩展配置）	6
4.5	在 docker-compose.yml 中追加管理端服务	7
4.6	启动顺序与健康检查	8
4.7	数据持久化与备份	8
5	非 Docker 源码部署	8
5.1	ML 推理服务	8
5.2	管理端后端	9
5.3	管理端前端	9
5.4	Nginx 反向代理示例	9
6	关键配置项说明	10
7	常见问题排查	10
7.1	前端打不开 / 白屏	10
7.2	登录后 401 / 实时数据不刷新	11
7.3	Torch 启动报错 CUDA out of memory 或 Illegal instruction	11
7.4	PostgreSQL 连接失败	11
7.5	告警与持续监控面板”没数据”	12
7.6	3D 场景卡顿 / WebGL 不可用	12
7.7	Docker 构建慢 / 拉取超时	12
7.8	SSE 连接 30 秒后被 Nginx 断开	12

7.9	Windows 开发环境 Python 包编译失败	13
8	附录	13
8.1	端口一览	13
8.2	健康检查命令速查	13
8.3	日志位置	13

1 项目概述

1.1 系统简介

智洁园区废气综合管理平台是一套面向工业园区废气治理的实时监测、智能预警与辅助决策系统。系统通过对现场 RTO（蓄热式热氧化炉）、转轮吸附装置、喷涂工艺段等关键工段的 26 项传感器数据进行秒级采集、分钟级预测，结合 AI 模型输出未来 6 小时 VOCs 浓度趋势、异常根因归因与 RAG（检索增强生成）处置建议，支撑运行人员、运维人员与管理人员三级协同处置。

1.2 子系统构成

子系统	目录	端口	职责
ML 推理服务	./ (根目录)	8001	传感器数据接入、LSTM/Seq2Seq 预测、SSE 实时推送
管理端后端	admin/backend/	8002	管理大屏聚合 API、JWT 鉴权、RAG、告警管理
管理端前端	admin/frontend/	5173	管理大屏 Vue3 + Three.js 可视化前端
用户端后端	client-backend/	8003	面向园区用户的 REST API
用户端前端	client-frontend/	—	uni-app 小程序 / H5
PostgreSQL	TimescaleDB (容器)	5432	告警历史、工单、RAG 计划持久化
Redis	Redis 7 (容器)	6379	热点缓存（可选）
Nginx	nginx-1.28.0/	5050	反向代理与静态资源

1.3 数据流概览

数据流

```
现场 PLC / DCS → POST /sensor-data → vocs_server (8001) → SSE /events  
→ admin-backend (8002) → GET /api/dashboard/overview + SSE /api/events →  
admin-frontend (5173)
```

2 环境配置要求

2.1 硬件最低配置

项目	最低配置（开发/演示）	推荐配置（生产）
CPU	4 核（x86_64）	8 核以上，支持 AVX2
内存	8 GB	16 GB 及以上
硬盘	40 GB SSD	100 GB SSD（RAID1）
GPU	非必须	可选 NVIDIA T4/A10 （加速 Torch 推理）
网络	千兆内网	千兆内网 + 公网备用链路

2.2 操作系统与基础软件

- **操作系统：**Ubuntu 22.04 LTS 或 CentOS 7.9 / Rocky Linux 9；Windows 10/11（仅限本地开发）
- **Docker Engine：**>= 24.0
- **Docker Compose：**v2.20 及以上（Compose V2 插件形式）
- **Python：**3.10（ML 服务）/ 3.11（管理端后端均可运行）
- **Node.js：**>= 18.18，推荐 20.x LTS
- **Nginx：**>= 1.24（非 Docker 部署时需要）
- **PostgreSQL：**14/15（推荐 timescale/timescaledb:latest-pg15）
- **时区：**Asia/Shanghai（容器 TZ 环境变量已显式声明）

注意：Torch 版本一致性

项目根目录 requirements.txt 固定 torch==2.7.0，admin/backend/requirements.txt 固定 torch==2.3.1。两者分属独立虚拟环境或独立容器，切勿合并安装到同一个 venv，否则会触发 CUDA 版本冲突与模型反序列化失败。

3 依赖包清单

3.1 ML 推理服务（根目录 requirements.txt）

包名	版本	说明
fastapi	0.104.1	Web 框架
uvicorn	0.24.0	ASGI 服务器
torch	2.7.0	深度学习推理，Seq2Seq 预测模型
numpy	>=1.26.0	数值计算

包名	版本	说明
pandas	$\geq 2.0.3$	时序数据处理
pydantic	2.4.0	数据校验与序列化
requests	2.31.0	同步 HTTP 客户端（测试脚本）
sseclient	0.0.27	SSE 测试客户端
scikit-learn	1.7.0	StandardScaler 反归一化

3.2 管理端后端（admin/backend/requirements.txt）

包名	版本	说明
fastapi	0.116.1	Web 框架
uvicorn	0.35.0	ASGI 服务器
httpx	0.28.1	异步 HTTP，代理到 <code>vocs_server</code>
pyjwt	2.10.1	管理员 JWT 颁发与校验
pydantic	2.11.7	数据模型
psycopg2-binary	2.9.10	PostgreSQL 驱动
python-dotenv	1.2.2	<code>.env</code> 加载
sentence-transformers	2.2.2	RAG 向量化
torch	2.3.1	向量模型推理
openai	2.31.0	大模型接口（RAG 生成端）
numpy	1.26.4	向量运算
pandas	3.0.2	结构化数据
pypdf	6.10.0	RAG 知识库 PDF 解析
python-docx	1.2.0	RAG 知识库 DOCX 解析

3.3 管理端前端（admin/frontend/package.json）

依赖	版本	说明
vue	$\sim 3.5.32$	核心框架
vue-router	$\sim 4.6.4$	路由
pinia	$\sim 3.0.2$	状态管理

依赖	版本	说明
axios	^1.15.0	HTTP 客户端
element-plus	^2.9.7	主题组件库（暗黑模式）
@element-plus/icons-vue	^2.3.1	图标集
echarts	^6.0.0	图表
vue-echarts	^8.0.1	ECharts Vue 封装
three	^0.183.2	3D 工厂可视化
dayjs	^1.11.13	时间格式化
vite	^5.4.19	构建工具 (dev)
typescript	~6.0.2	类型系统 (dev)
vue-tsc	^3.2.6	Vue 类型检查 (dev)

4 Docker 部署步骤

4.1 目录准备

```
# 1. 克隆代码
git clone <your-repo-url> waste-gas
cd waste-gas

# 2. 目录结构确认
ls -al
# 应包含: vocs_server.py  admin/  backend/  client-backend/
#          Dockerfile  docker-compose.yml  models/  requirements.txt
```

4.2 编写/确认环境变量文件

在项目根目录创建 `.env`（不要提交到 **Git**），生产环境请替换为强密码：

```
# ===== 管理端后端 =====
ADMIN_JWT_SECRET=please-change-me-in-production
ADMIN_USERNAME=admin
ADMIN_PASSWORD=YourStrongPassword!2026
ADMIN_TOKEN_EXPIRE_MINUTES=120

# ===== 数据库 =====
PG_HOST=postgres
PG_PORT=5432
PG_DB=aqimonitor
PG_USER=team
```

```
PG_PASSWORD=YourDbPassword!2026
PG_POOL_MIN=1
PG_POOL_MAX=8

# ===== ML 服务 =====
MODEL_PATH=models/vocs_seq2seq_v2_best.pth
SCALER_PATH=models/vocs_scalers_v2.pkl
CSV_PATH=vocs_realtime_data/vocs_realtime_data.csv
TZ=Asia/Shanghai

# ===== RAG (可选) =====
OPENAI_API_KEY=sk-xxx
OPENAI_BASE_URL=https://api.openai.com/v1
```

4.3 一键启动全部服务

项目根目录已提供 Dockerfile 与 docker-compose.yml。默认仅启动 ML 推理服务；管理端、用户端与数据库请按需启用。

```
# 构建镜像（首次/代码变更后执行）
docker compose build

# 启动 ML 推理服务（8001）—— 仅启动 vocs-server
docker compose up -d vocs-server

# 同时启动 Postgres + Redis 基础设施（profiles: infra）
docker compose --profile infra up -d

# 查看容器状态
docker compose ps

# 查看日志
docker compose logs -f vocs-server
```

4.4 管理端独立镜像（推荐扩展配置）

当前仓库内 docker-compose.yml 默认只打包 vocs_server。生产环境建议在 admin/backend 与 admin/frontend 下分别新建 Dockerfile，并在 compose 中加入对应服务定义：

```
# admin/backend/Dockerfile
FROM python:3.11-slim
WORKDIR /app
ENV PYTHONUNBUFFERED=1 PYTHONDONTWRITEBYTECODE=1 TZ=Asia/Shanghai
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential libpq-dev && rm -rf /var/lib/apt/lists/*
COPY admin/backend/requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
COPY admin/backend/ ./
EXPOSE 8002
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8002"]
```

```
# admin/frontend/Dockerfile (多阶段构建)
FROM node:20-alpine AS build
WORKDIR /app
COPY admin/frontend/package*.json ./
RUN npm ci
COPY admin/frontend/ ./
RUN npm run build

FROM nginx:1.27-alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY admin/frontend/nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

4.5 在 docker-compose.yml 中追加管理端服务

```
admin-backend:
  build:
    context: .
    dockerfile: admin/backend/Dockerfile
  container_name: admin-backend
  ports: ["8002:8002"]
  env_file: [".env"]
  environment:
    - VOCS_BASE_URL=http://vocs-server:8001
  depends_on: [vocs-server, postgres]
  restart: unless-stopped
  networks: [vocs-network]

admin-frontend:
  build:
    context: .
    dockerfile: admin/frontend/Dockerfile
  container_name: admin-frontend
  ports: ["5173:80"]
  depends_on: [admin-backend]
  restart: unless-stopped
  networks: [vocs-network]
```

4.6 启动顺序与健康检查

1. 启动数据库与缓存: `docker compose --profile infra up -d postgres redis`
2. 等待数据库就绪 (约 15 秒): `docker compose logs postgres | grep "ready to accept"`
3. 启动 ML 服务: `docker compose up -d vocs-server`
4. 观察 healthcheck: `docker inspect --format "{{.State.Health.Status}}" vocs-control-system`
5. 启动管理端: `docker compose up -d admin-backend admin-frontend`
6. 浏览器访问: <http://< IP>:5173>

默认账户

管理员用户名: admin 默认密码: admin123456

首次登录后请立即通过 `.env` 的 `ADMIN_PASSWORD` 修改并重启 `admin-backend`。

4.7 数据持久化与备份

- 实时 CSV 回放数据: 卷挂载 `./vocs_realtime_data` → `/app/vocs_realtime_data`
- 模型文件: 卷挂载 `./models` → `/app/models` (用于热更新)
- PostgreSQL 数据: 命名卷 `postgres_data`; 建议每日 `pg_dump` 备份
- Redis 数据: 命名卷 `redis_data`, AOF 持久化已开启

```
# 数据库每日备份 (建议配合 cron)
docker exec vocs-postgres pg_dump -U team -d aqimonitor \
  | gzip > backup/pg_$(date +%Y%m%d).sql.gz
```

5 非 Docker 源码部署

适用于本地开发调试、答辩演示或无法使用 Docker 的现场环境。

5.1 ML 推理服务

```
# Windows PowerShell 示例 (Linux 将 python 换成 python3、venv 激活脚本换成 source)
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python vocs_server.py
# 或
uvicorn vocs_server:app --host 0.0.0.0 --port 8001
```

5.2 管理端后端

```
cd admin/backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
# 设置环境变量 (Linux 用 export)
$env:ADMIN_PASSWORD = "YourStrongPassword"
$env:VOCS_BASE_URL = "http://127.0.0.1:8001"
uvicorn main:app --host 0.0.0.0 --port 8002 --reload
```

5.3 管理端前端

```
cd admin/frontend
npm install           # 或 npm ci (基于 package-lock.json)
npm run dev           # 开发模式, 监听 5173
# 生产构建
npm run build
# 产物位于 admin/frontend/dist, 使用 nginx 托管
```

5.4 Nginx 反向代理示例

```
server {
    listen 5050;
    server_name _;

    # 管理端前端静态资源
    location / {
        root /opt/waste-gas/admin/frontend/dist;
        try_files $uri $uri/ /index.html;
    }

    # 管理端 API
    location /api/ {
        proxy_pass http://127.0.0.1:8002;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    # ML 服务 SSE (注意关闭缓冲)
    location /vocs/ {
        proxy_pass http://127.0.0.1:8001/;
        proxy_buffering off;
        proxy_cache off;
    }
}
```

```

        proxy_set_header Connection "";
        proxy_http_version 1.1;
        chunked_transfer_encoding on;
    }
}

```

6 关键配置项说明

环境变量	默认值	说明
ADMIN_JWT_SECRET	waste-gas-admin-secret	JWT 签名密钥，生产必须更换
ADMIN_USERNAME	admin	管理员登录名
ADMIN_PASSWORD	admin123456	管理员登录密码，生产必须更换
ADMIN_TOKEN_EXPIRE_MINUTES	120	Token 有效期（分钟）
VOCS_BASE_URL	http://127.0.0.1:8001	ML 服务地址，容器内用服务名
VOCS_REQUEST_TIMEOUT	4	HTTP 请求超时（秒）
ENSEMBLE_BASE_URL	http://127.0.0.1:8000	集成模型服务（可选）
ADMIN_TOTAL_EQUIPMENT	150	设备总数（展示用）
PG_HOST / PG_PORT	98.142.241.155 / 5432	默认指向开发库，生产必须覆盖
PG_DB / PG_USER / PG_PASSWORD	aqimonitor / team / ...	数据库凭据
MODEL_PATH	models/vocs_seq2seq_v2_order	模型路径
SCALER_PATH	models/vocs_scalers_v2_pk	scaler 路径
TZ	Asia/Shanghai	时区，影响日志与告警时间戳

7 常见问题排查

7.1 前端打不开 / 白屏

现象

浏览器访问 `http://<IP>:5173` 或 `http://<IP>:5050` 空白，或控制台报 `Failed to load resource: net::ERR_CONNECTION_REFUSED`。

排查步骤：

1. 确认容器存活: `docker compose ps`, STATUS 列应为 Up (healthy)。
2. 确认端口监听: `ss -ltnp | grep -E '5173|5050|8002'`。
3. 确认防火墙放行: `ufw status / firewall-cmd --list-ports`。
4. 检查 Nginx 配置 root 指向的 dist 目录是否已生成 (`npm run build`)。
5. 前端 `vite.config.ts` 中代理目标是否与后端实际端口一致。

7.2 登录后 401 / 实时数据不刷新

可能原因:

- JWT 过期: 检查 `ADMIN_TOKEN_EXPIRE_MINUTES`, 登录后重新获取 token。
- SSE 被 Nginx 缓冲: 确认 Nginx 已关闭 `proxy_buffering`、`proxy_cache`, 且 `proxy_read_timeout` $\geq 3600s$ 。
- `vocs-server` 未启动或健康检查失败: `docker logs vocs-control-system`。
- 浏览器控制台 `/api/events` 连接断开: 检查反向代理是否支持 `chunked` 传输。

7.3 Torch 启动报错 CUDA out of memory 或 Illegal instruction

CPU 不支持 AVX2 时

某些旧服务器缺少 AVX2 指令集, PyTorch 2.7.0 默认二进制会在加载时触发 `Illegal instruction (core dumped)`。解决: 降级到 `torch==2.0.1+cpu` 或改用 ONNX Runtime 部署。

排查:

1. `cat /proc/cpuinfo | grep -o 'avx2' | head -1` 验证 CPU 指令集。
2. `nvidia-smi` 查看显存占用; 将 `torch.load` 切换 `map_location="cpu"`。
3. 检查 `models/` 目录是否成功挂载到容器 `/app/models`。

7.4 PostgreSQL 连接失败

典型报错: `psycopg2.OperationalError: could not connect to server`

排查:

1. 容器内用服务名 `postgres` 而非 `127.0.0.1`——共享 compose 网络后 `PG_HOST=postgres`。
2. 外部访问需确认 `postgresql.conf` 的 `listen_addresses='*'` 与 `pg_hba.conf`。
3. 云库: 白名单是否放行当前出口 IP。
4. 应用启动日志出现 `[DB] WARNING: init_pool failed`——RAG 持久化功能将自动降级但不阻塞启动。

7.5 告警与持续监控面板”没数据”

- 前端已内置 mock 演示数据（stores/alerts.ts、stores/dashboard.ts），若连接完全断开也会回退显示；若 mock 也不出现，多为前端构建产物未更新。
- 确认 vocs_server 有数据接入：curl http://127.0.0.1:8001/alerts?limit=5。
- 管理端后端日志 httpx.ConnectError：检查 VOCS_BASE_URL 是否匹配实际网络。

7.6 3D 场景卡顿 / WebGL 不可用

- 浏览器需启用硬件加速；集成显卡建议窗口分辨率 $\leq 1920 \times 1080$ 。
- about:gpu (Chrome) 确认 WebGL2 处于 Hardware accelerated。
- 远程桌面/RDP 下常见 WebGL fallback 到软件渲染，建议现场演示用本机浏览器。

7.7 Docker 构建慢 / 拉取超时

```
# /etc/docker/daemon.json 配置国内镜像加速
{
  "registry-mirrors": [
    "https://docker.mirrors.ustc.edu.cn",
    "https://hub-mirror.c.163.com"
  ]
}
sudo systemctl restart docker

# pip 国内源
pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple

# npm 国内源
npm config set registry https://registry.npmirror.com
```

7.8 SSE 连接 30 秒后被 Nginx 断开

Nginx 默认 proxy_read_timeout 为 60 秒，长连 SSE 会被动关闭后前端每 3 秒重连。修改如下：

```
proxy_read_timeout 3600s;
proxy_send_timeout 3600s;
proxy_http_version 1.1;
proxy_set_header Connection "";
proxy_buffering off;
```

7.9 Windows 开发环境 Python 包编译失败

典型:psycopg2-binary / sentence-transformers 编译 C 扩展时报缺少 Microsoft Visual C++ 14.0。解决方案:

- 安装 **Visual Studio Build Tools**, 勾选”Desktop development with C++”;
- 或直接使用 Docker 部署, 绕开原生编译。

8 附录

8.1 端口一览

端口	服务	用途
8001	vocs-server	ML 推理 REST + SSE
8002	admin-backend	管理端聚合 API + JWT
8003	client-backend	用户端 REST
5173	admin-frontend (dev)	Vite 开发服务器
5050	nginx	反向代理 / 静态资源
5432	postgres	时序数据库
6379	redis	缓存

8.2 健康检查命令速查

```
# ML 服务
curl -s http://127.0.0.1:8001/status | python -m json.tool

# 管理端后端
curl -s http://127.0.0.1:8002/api/health

# PostgreSQL
docker exec vocs-postgres pg_isready -U team -d aqimonitor

# 全量容器健康
docker ps --format 'table {{.Names}}\t{{.Status}}'
```

8.3 日志位置

- Docker: `docker compose logs -f <service>`

- 非容器: backend/backend-stdout.log、backend/backend-stderr.log
- Nginx: nginx-1.28.0/logs/access.log、error.log

技术支持

如排查未果, 请联系项目组并提供:

1. docker compose ps 截图
2. 问题服务最近 200 行日志
3. 浏览器控制台 Network 面板截图 (如为前端问题)
4. 操作系统、Docker、Python、Node 版本信息

— 文档结束 —

智洁园区项目组 · 2026